Decision Structures

CSCI-SHU 11
Fall 2025

Lecture's Outline

- Booleans
  - comparison operators (==, !=, >, >=, <, <=)
  - membership operators (in, not in)
  - logical operators (not, and, or)
- Decision structure
  - *if* statement
  - *if-else* statement
  - *if-elif-else* statement
  - nested *if* statement

Booleans

Boolean

- Boolean is a basic data type
  - as data types:
    - int, float, complex, string
  - only two values are booleans:
    - *True* or *False*
  - they can be assigned to variables

```
>>> a = True
>>> type(a)
<class 'bool'>

>>> b = False
>>>type(True)
<class 'bool'>
```

Comparison Operators

- Boolean two values are returned by 6 comparison operators

Comparison operators	Meaning
a == b	is a equal to b?
a != b	is a different from b?
a > b	is a greater than b?
a < b	is a lower than b?
a >= b	is a greater than or equal to b?
a <= b	is a lower than or equal to b?

Comparison Operators

Numeric operands

- Boolean two values are returned by 6 comparison operators

a	*b*	Comparison operators	Result
1	2	`a == b`	`False`
1	2	`a != b`	`True`
1	2	`a > b`	`False`
1	2	`a < b`	`True`
1	2	`a >= b`	`False`
1	2	`a <= b`	`True`

Comparison Operators

Numeric operands

- Mixed numeric types can be compared

a	*b*	Comparison operators	Result
`1`	`2.0`	`a == b`	`False`
`1`	`2.0`	`a != b`	`True`
`1.0`	`2`	`a > b`	`False`
`1.0`	`2`	`a < b`	`True`
`1.0`	`2.0`	`a >= b`	`False`
`1.0`	`2.0`	`a <= b`	`True`

Comparison Operators

String operands

- Strings can be also compared
  - Lexicographical ordering for strings uses the ASCII ordering for individual characters.

```
>>> ord('a')
    97

>>> ord('b')
    98

>>> 'a' < 'b'
    True

>>> 'a'== 'a'
    True

>>> 'aa' < 'ab'
    True

>>> 'aba' > 'aab'
    True
```

ASCII Table

Char	Dec	Char	Dec	Char	Dec
0	48	A	65	a	97
1	49	B	66	b	98
...		...		...	
9	57	Z	90	z	122

Comparison Operators

Mixing Strings and Numeric values

- Comparing a string with a numeric value will result in:
  - == will always return False
  - != will always return True
  - <, >, <=, and >= will raise a Type Error

```
>>> 'a' == 2
    False

>>> 'a' != 2
    True

>>> 'a' < 2
Traceback (most recent call last):
  File "<pyshell#266>", line 1, in <module>
    'a' < 2
TypeError: '<' not supported between instances of 'str' and 'int'

>>> 'a' >= 2
Traceback (most recent call last):
  File "<pyshell#266>", line 1, in <module>
    'a' < 2
TypeError: '<' not supported between instances of 'str' and 'int'
```

Booleans as Numbers

- Booleans are considered a numeric type
  - *True* is equal to 1 and *False* is equal to 0
  - arithmetic operations apply to Booleans

```
>>> True == 1
    True

>>> False == 0
    True

>>> True > False
    True

>>> False > True
    False

>>> True + (False / True)
    1.0
```

Membership Operator

- Check if an element is in a sequence
  - *in* and *not in* operators can take two arguments

```
>>> 'a' in "abc"
    True

>>> 'a' in "abc"
    True

>>> 'A' not in "abc"
    True

>>> "ab" in "abc"
    True

>>> "ac" not in "abc"
    True
```

Logical Operators

- Logical operators are operators that combine Boolean values
  - these operators always return another Boolean value
  - these operators can be use to create complex Boolean expressions
- 3 logical operators:

Logical operator	Meaning
`not a`	only one operand on the right returns True if a is False
`a or b`	two operands (one on each side) returns True if at least one operand is True
`a and b`	two operands (one on each side) returns True if both operands are True

Logical Operators

Truth Tables

a	not *a*
False	True
True	False

and	False	True
False	False	False
True	False	True

or	False	True
False	False	True
True	True	True

Precedence

- If no parentheses, Logical Operators are evaluated with the following order of priority:
  - not
  - and
  - or
- Example:
  - not a or not b and c
  - (not a) or ((not b) and c)

Combining Operators

Summary of Operator Precedence

- List of operators from highest to lowest precedence:

Operators	Meaning
( )	parentheses are evaluated first
**	exponentiation
+, –	unary + and - (signs)
*, /, //, %	multiplication, division(s) and modulo
+, –	addition and subtraction
==, !=, <, <=, >=, >	comparison operators
not	
and	
or	

Chaining Operators

```
100 > 1 and -10 ** 2 <= -100 or "foo" == "foo" + "bar"
```

Chaining Operators

```
True and False or False and True and 10 * 10 + 10 == 110 and not False
```

Combining Operators

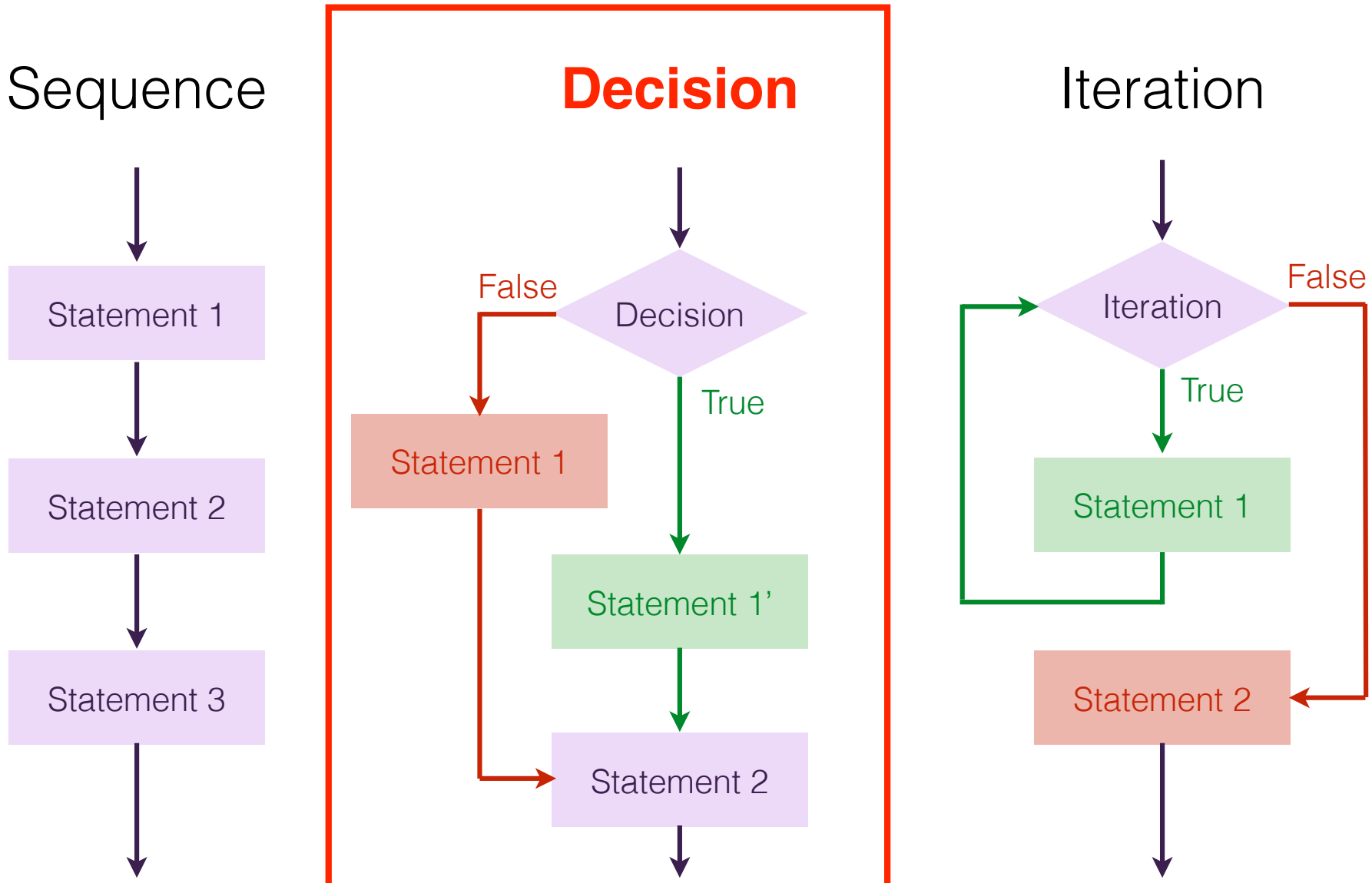
Short Circuit Evaluation

- *or* and *and* operators have particular properties
  - Let's suppose *unknown* is a boolean variable...
  - …but its value is unknown
    - *True or unknown* always returns *True*
    - *False and unknown* always returns *False*
    - ...whatever the value of *unknown* (*True* or *False*)
- **True and True** or False and True and 10 * 10 + 10 == 110 and not False
  - always return *True*

Decision Structure

The *if elif else* statements

Control Structures



Decision Structure

- The *if* Statement:
  - allows for conditional execution of code
  - allows a program to have more than one path of execution
  - executes one or more statements only when a Boolean is True

```
night = True

if night:
  sky = "Dark"
else:
  sky = "Blue"
```

night	True	False
sky	"Dark"	"Blue"

```
pill = input("Choose a pill (blue/red): ")

if pill == 'blue':
    print("The story ends.")
else:
    print("You stay in Wonderland.")
```

The *if* Statement Synthax

- the *if* Statement:
  - starts with keyword *if*
  - followed by a boolean expression
  - which ends with a colon *:*
  - the code to be executed if the condition is met is placed in an indented code block
  - once the indentation goes back one level, the body of the if-statement ends

```
if <boolean expression>:
   # Statements to execute if
   # condition is true
```

The *if* Statement Synthax

- the *if* Statement:
  - starts with keyword *if*
  - followed by a boolean expression
  - which ends with a colon *:*
  - the code to be executed if the condition is met is placed in an indented code block
  - once the indentation goes back one level, the body of the if-statement ends

```
number = int(input('Enter a number: '))

if number >= 0:
   print(number, "is positive.")

print("That's all folks.")
```

The *if-else* Statement

- The *if-else* Statement:
  - for dual alternative decision structure
    - one block of statements if the condition is True
    - another block of statements is executed if the condition is False
- The *else* Statement:
  - After the body of an if-statement
  - go back one level of indentation to mark that the previous code
  - keyword else , immediately followed by a colon *:*
  - the code to be executed if the condition is not met is placed in an indented code block
  - once the indentation goes back one level, the body of the else-statement ends

```
number = int(input('Enter a number: '))

if number >= 0:
    print(number, "is positive.")
else:
    (number, "is strictly negative.")

print("That's all folks.")
```

The *if-else* Statement

Condition is True

```
number = 10

if number > 0:
    print(number, "is positive")
else:
    print(number, "is strictly negative")

print("That's all folks.")
```

Condition is False

```
number = -5

if number > 0:
    print(number, "is positive")
else:
    print(number, "is strictly negative")

print("That's all folks.")
```

The *if-else* Statement

- Write a program to ask the user for 2 numbers:
  - if they are equal, print the sum
  - if they are different, print the product

The *if-elif-else* statement

- The *elif* statement syntax
  - After the body of an if-statement…
  - go back one level of indentation to mark that the previous code block has ended
  - keyword *elif*, immediately followed by another condition and a colon *:*
  - body - indented, body ends when indentation goes back one level
  - as many *elif*-statements as needed. . .
  - use *else*-statement at the end if needed

```
if condition1:
    # code block 1
elif condition2:
    # code block 2
elif ...

else:
    # code block 3
```

The *if-elif-else* statement

- In a if-elif-else statement:
  - The body of the first *True* condition will be executed
    - even if several conditions (if or elif) are *True*
    - order matters
- If none of the conditions is True
  - the body of the *else* clause will be executed (if present)
- Write a program that prompts the user for a number:
  - print "Positive" if the number is positive
  - print "Negative" if the number is negative
  - print "Zero" if the number is 0

The *if-elif-else* statement

- In a if-elif-else statement:
  - The body of the first *True* condition will be executed
    - even if several conditions (if or elif) are *True*
    - order matters
- If none of the conditions is True
  - the body of the *else* clause will be executed (if present)
- Write a program that prompts the user for a number:
  - print "Positive" if the number is positive
  - print "Negative" if the number is negative
  - print "Zero" if the number is 0

```
number = int(input('Enter a number: '))

if number > 0:
    print(number, "is strictly positive.")
elif number < 0:
    print(number, "is strictly negative.")
else: # number == 0
    print(number, "is zero.")

print("This is the end of the program.")
```

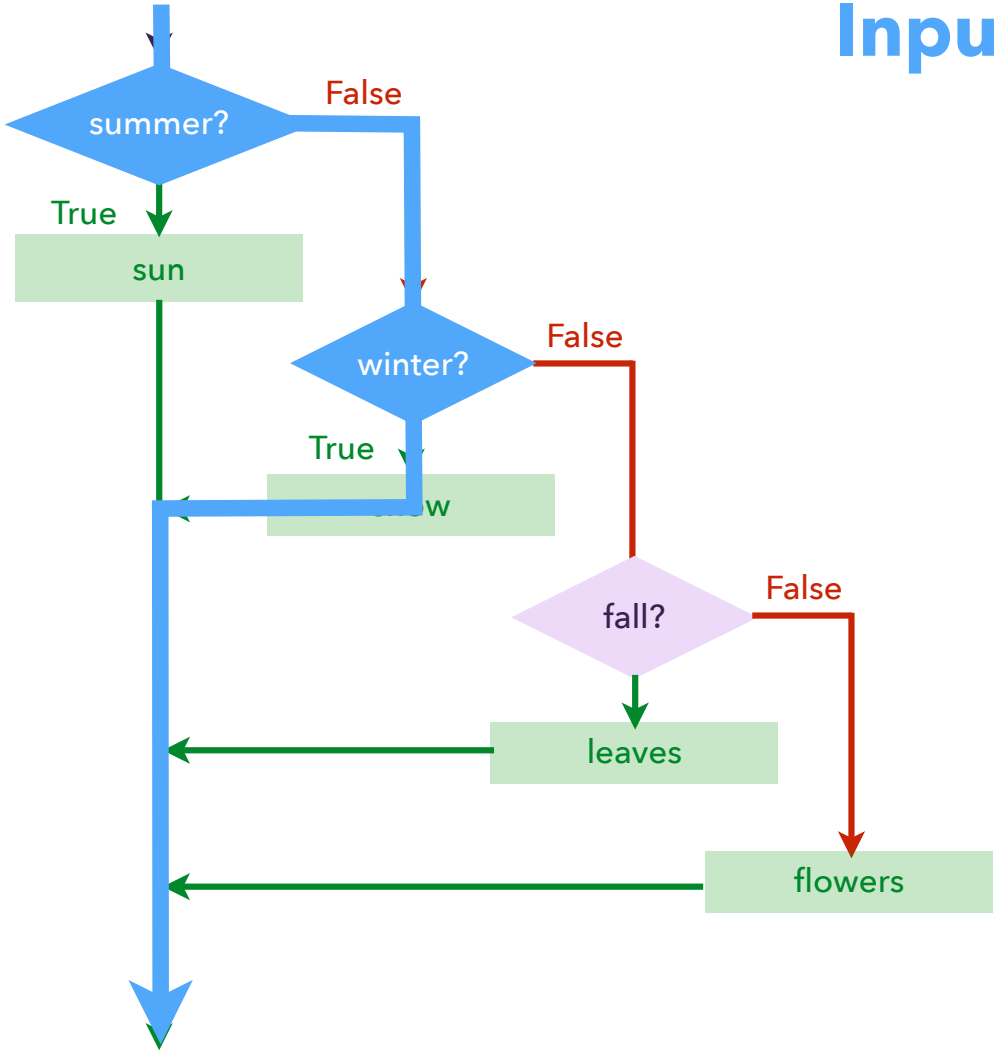
The *if-elif-else* statement

```
season = input("Enter a season: ")

if season == "Summer":
  print("Sun")
elif season == "Winter":
  print("Snow")
elif season == "Fall":
  print("Leaves")
else:   # season is Spring
  print("Flowers")
```

Season	Summer	Winter	Fall	Spring
Output	Sun	Snow	Leaves	Flowers

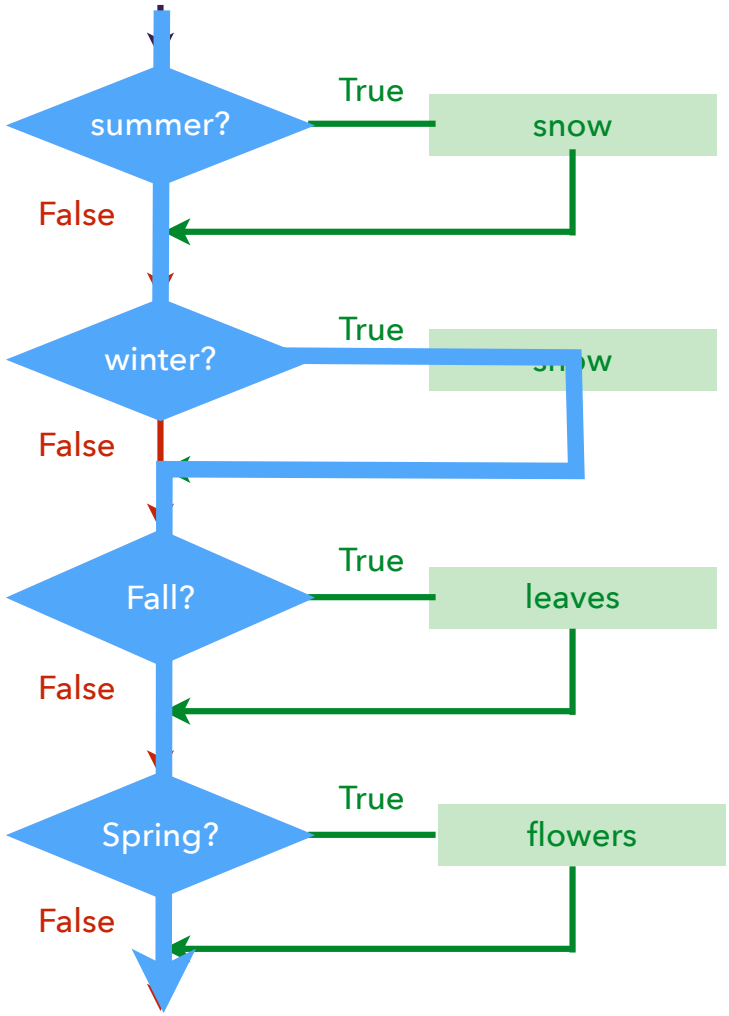
The *if-elif-else* statement



Input : winter

```
if season == "Summer":
  print("Sun")
elif season == "Winter":
  print("Snow")
elif season == "Fall":
  print("Leaves")
else:   # season is Spring
  print("Flowers")
```

The *if-elif-else* statement

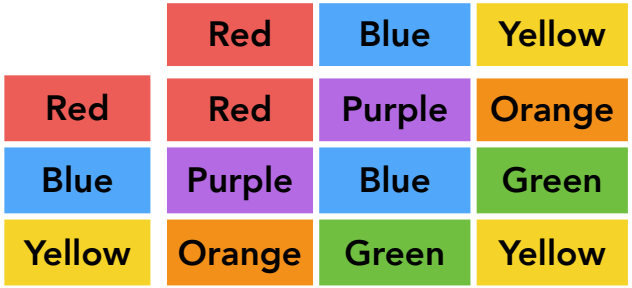


Input : winter

```
if season == "Summer":
  print("Sun")
if season == "Winter":
  print("Snow")
if season == "Fall":
  print("Leaves")
if season == "Spring":
  print("Flowers")
```

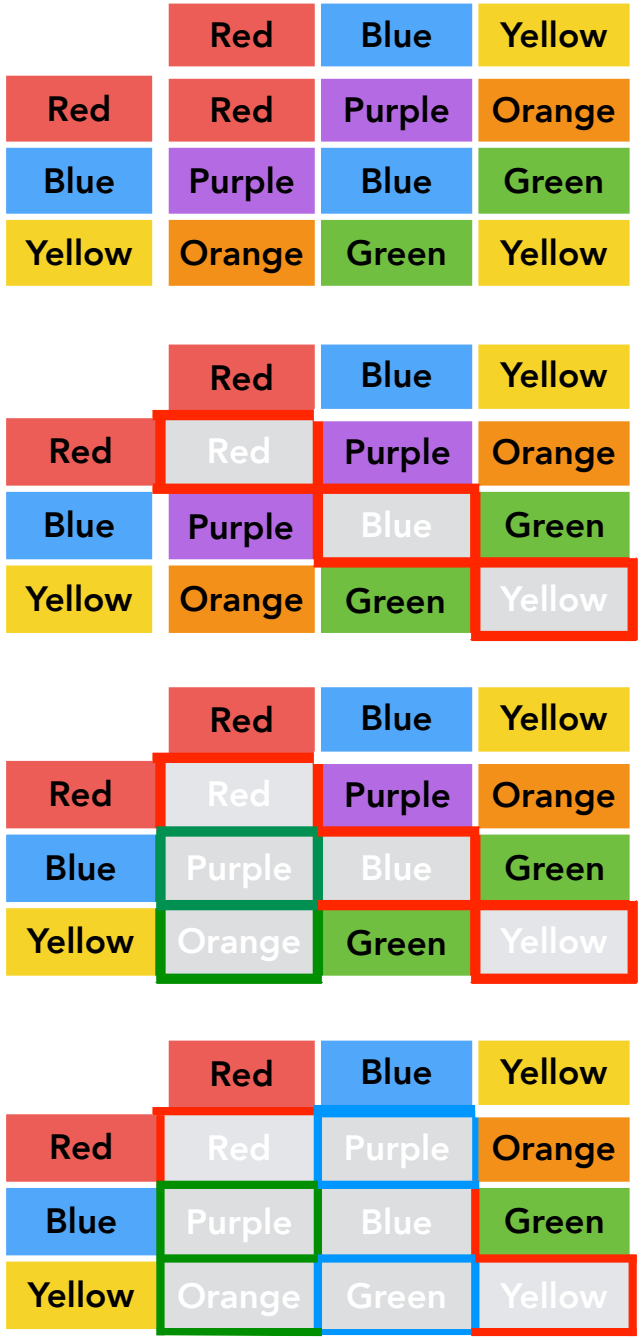
Color Mixer

```
color_1 = input("Input color 1: ")
color_2 = input("Input color 2: ")
```



```
color_1 = input("Input color 1: ")
color_2 = input("Input color 2: ")

if color1 == color 2:
  print("Color is", color_1)
elif color_1 == "Red":
  if color_2 == "Blue":
    print("Color is Purple")
  else:  # color_2 is Yellow
    print("Color is Orange")
elif color_1 == "Blue":
  if color_2 == "Red":
    print("Color is Purple")
  else: # color_2 is Yellow
    print("Color is Green")
else: # color_1 is Yellow
  if color_2 == "Red":
    print("Color is Orange")
  else: # color_2 is Blue
    print("Color is Blue")
```



Takeways

- Boolean variables and expressions
  - either True or False
  - formed using comparison/membership operators
- Decision structure
  - a program can have different paths of execution (logical branches)
  - each path/branch depends on the evaluation of one or more conditions
  - simple decisions are implemented with the *if* statement
  - two-way decisions are implemented with the *if-else* statement
  - multi-way decisions are implemented with the *if-elif-else* statement

Readings & Assignments

- Assignments (deadline: Tue 09/23 09:45am):
  - Quizz 02
  - Quizz 03
- Preparation for Recitation 04:
  - Read chapter 3 in the textbook
  - Watch self-paced modules 3:
    - https://stream.nyu.edu/playlist/dedicated/306991832/1_gm6t0b7p/
  - Read and solve Recitation 04 question sheet
- Preparation for Lecture 05 (Tue 09/23):
  - Read chapter 4 in the textbook (you can skip 4.3)
  - Watch self-paced module 4:
    - https://stream.nyu.edu/playlist/dedicated/306991832/1_ullmbeie/